LAPORAN PRAKTIKUM 7
OVERRIDING DAN OVERLOADING

Disusun Oleh:
KHOHAR MUHAMAD FATAHURROHMAN
254107020021
Kelas TI 2B



PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
Oktober 2026

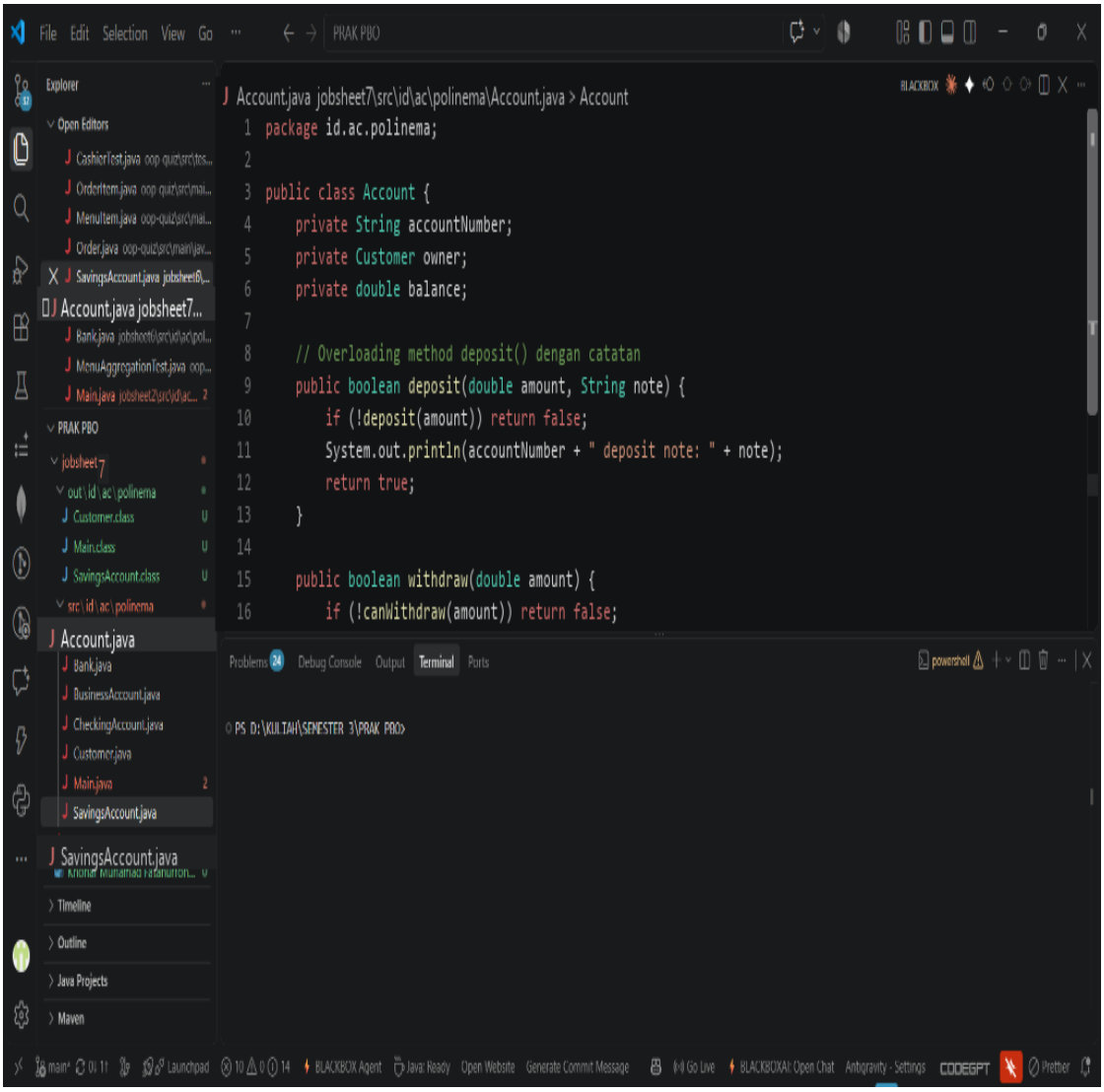
Langkah Percobaan

a. Struktur Project

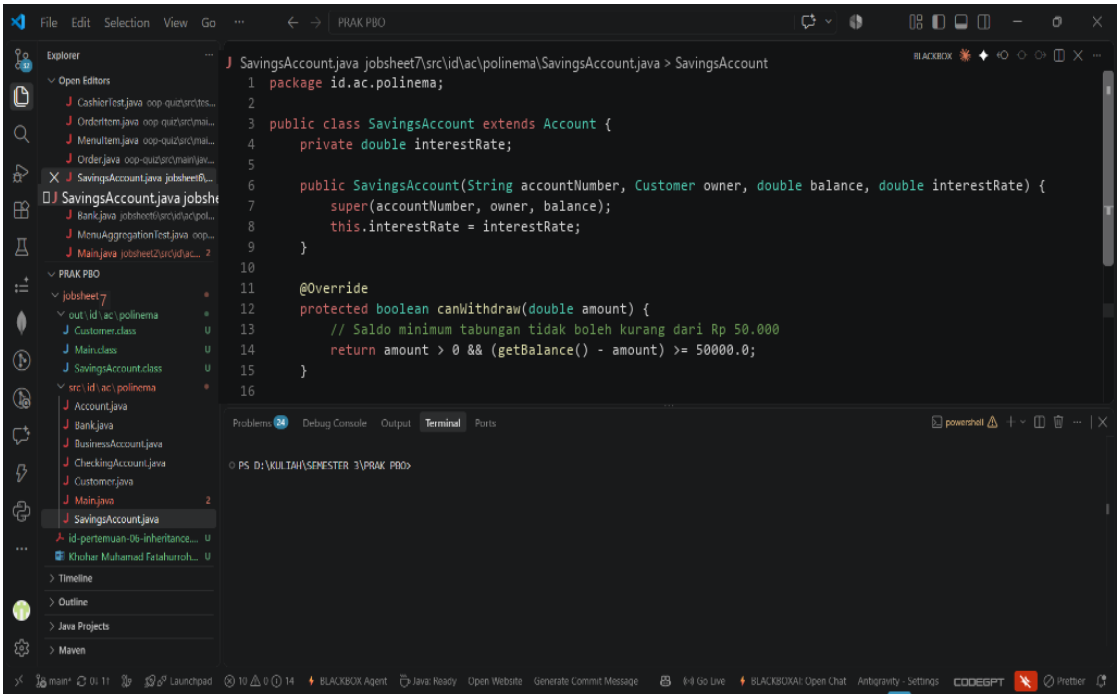
Proyek pada praktikum pertemuan ke-7 ini merupakan kelanjutan dari proyek bank-mini pada topik Inheritance, yang diorganisasikan ke dalam struktur package id.ac.polinema. Fokus utama pada pertemuan ini adalah menerapkan konsep Method Overriding (penulisan ulang method turunan dengan signature identik) dan Method Overloading (pembuatan beberapa method bernama sama namun dengan daftar parameter berbeda).

Account.java: Sebagai superclass, direfaktor dengan memisahkan logika validasi penarikan ke method protected boolean canWithdraw(double amount) sebagai titik override, serta menambahkan method deposit(double amount, String note) sebagai bentuk overloading.
SavingsAccount.java: Meng-override canWithdraw(double) untuk menetapkan saldo mengendap minimum sebesar Rp 50.000, serta meng-override printInfo() untuk menampilkan rincian suku bunga tabungan.
CheckingAccount.java: Meng-override canWithdraw(double) untuk mengizinkan penarikan melebihi saldo hingga batas toleransi overdraftLimit, serta meng-override printInfo() untuk menampilkan limit overdraft rekening giro.
BusinessAccount.java: Sebagai tugas mandiri, meng-override canWithdraw(double) dengan aturan saldo minimum Rp 1.000.000, serta meng-override printInfo() untuk menampilkan biaya administrasi bulanan rekening bisnis.

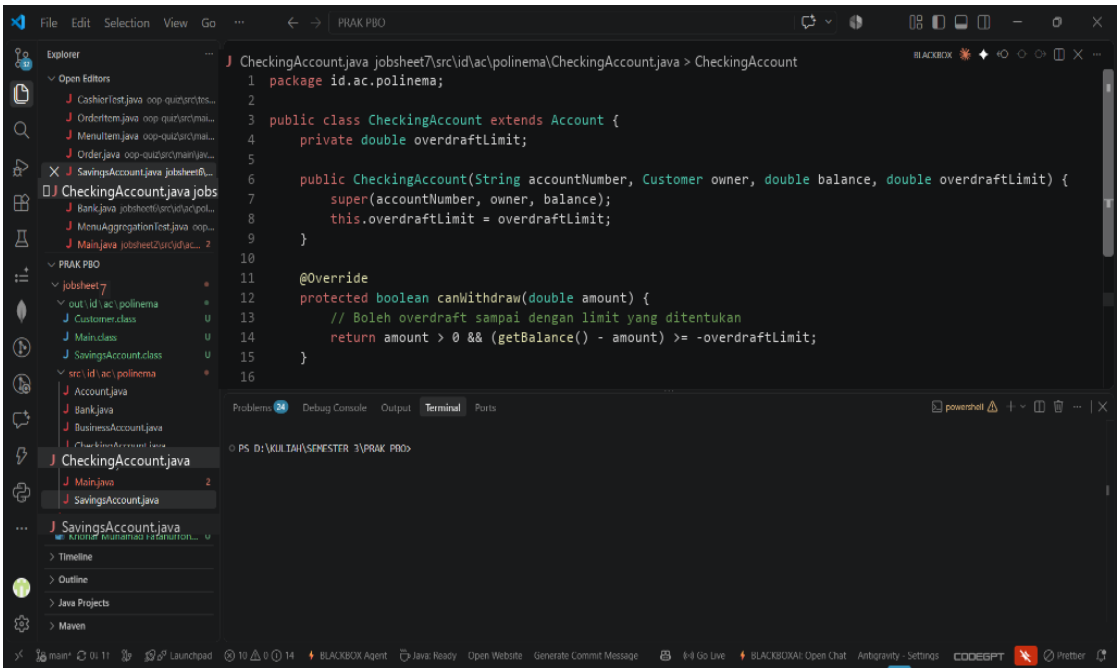
b. Implementasi & Menjalankan Program



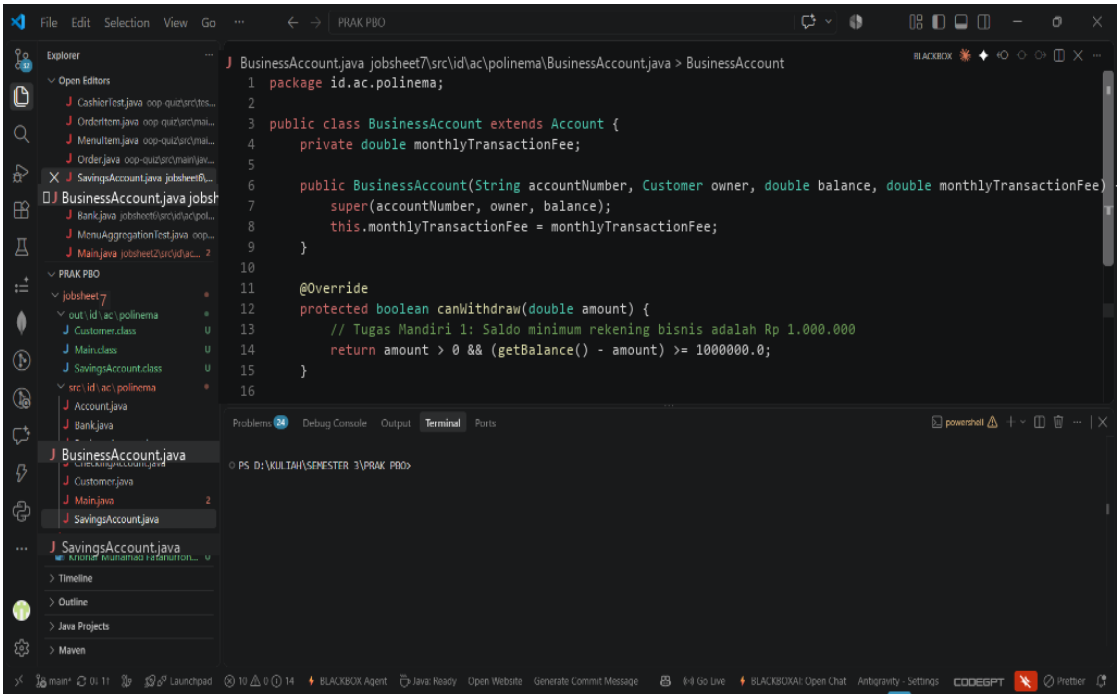
Gambar 1: Implementasi canWithdraw() dan Overloading deposit() pada Account.java



Gambar 2: Overriding canWithdraw() dan printInfo() pada SavingsAccount.java



Gambar 3: Overriding canWithdraw() dan printInfo() pada CheckingAccount.java



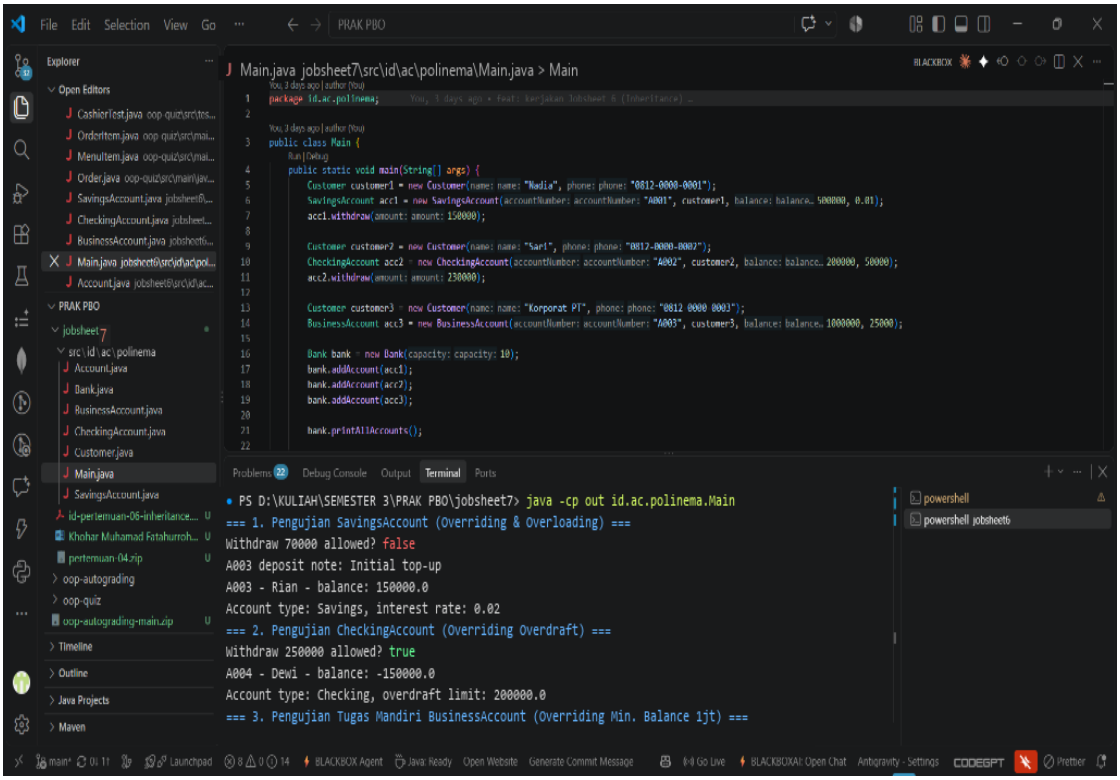
Gambar 4: Overriding canWithdraw() dan printInfo() pada BusinessAccount.java (Tugas Mandiri)



Gambar 5: Output Eksekusi Langkah 4 (Pengujian deposit() dengan Catatan)



Gambar 6: Output Eksekusi Tugas Mandiri (Uji Penarikan BusinessAccount Ditolak)



Gambar 7: Output Eksekusi Program Lengkap dan Pengujian Polimorfisme pada Bank

Penjelasan Kode, Konsep OOP, dan Kendala:

1. Method Overriding: Pada langkah 1 sampai 3, kita menerapkan teknik refactoring dengan mengekstraksi aturan validasi saldo ke dalam method protected boolean canWithdraw(double amount). Hal ini memberikan ekstensi fleksibel bagi setiap subclass untuk mendefinisikan aturan penarikannya sendiri tanpa merusak alur transaksi dasar pada method withdraw(double amount). Di subclass SavingsAccount, CheckingAccount, dan BusinessAccount, method canWithdraw dan printInfo ditandai dengan anotasi @Override. Anotasi ini berfungsi sebagai instruksi eksplisit kepada compiler Java untuk memverifikasi kesesuaian signature method turunan terhadap method induknya, mencegah terjadinya kesalahan pengetikan nama atau parameter. Selain itu, pemanggilan super.printInfo() di dalam method printInfo() subclass menunjukkan pemanfaatan kembali (code reuse) fungsionalitas superclass sebelum menambahkan informasi spesifik milik masing-masing rekening.

2. Method Overloading: Pada langkah 4, method deposit(double amount, String note) ditambahkan pada kelas Account. Method ini memiliki nama yang sama persis dengan deposit(double amount), namun memiliki daftar parameter yang berbeda (dua parameter vs satu parameter). Konsep ini disebut Method Overloading (Polimorfisme Statis / Compile-Time Polymorphism). Compiler menentukan method mana yang akan dieksekusi berdasarkan jumlah dan tipe data argumen saat pemanggilan. Di dalam method deposit berparameter ganda, kita mendelegasikan penambahan saldo kepada method deposit(amount) asli, sehingga aturan validasi nilai positif tidak perlu ditulis ulang (prinsip DRY - Don't Repeat Yourself).

3. Kendala dan Solusi: Kendala yang dihadapi saat menulis canWithdraw() pada subclass adalah atribut balance di superclass Account bersifat private. Subclass tidak dapat mengakses balance secara langsung. Solusinya adalah mengakses saldo menggunakan method getter publik getBalance() yang telah disediakan di superclass. Hal ini tetap mempertahankan prinsip enkapsulasi secara murni.

Hasil & Pembahasan

Berdasarkan seluruh hasil pengujian program konsol bank-mini, konsep overriding dan overloading berjalan dengan sangat baik dan akurat sesuai spesifikasi:

Uji Saldo Minimum SavingsAccount: Pada rekening A003 dengan saldo Rp 100.000, penarikan sebesar Rp 70.000 berhasil ditolak (mengembalikan false). Saldo akhir tetap Rp 100.000 karena sisa saldo setelah penarikan adalah Rp 30.000, yang melanggar batas saldo minimum tabungan yaitu Rp 50.000.

Uji Overdraft CheckingAccount: Pada rekening A004 dengan saldo Rp 100.000 dan batas overdraft Rp 200.000, penarikan sebesar Rp 250.000 berhasil diizinkan (true). Saldo berhasil berkurang menjadi negatif (-Rp 150.000) karena masih dalam rentang plafon pinjaman giro.

Uji Tugas Mandiri BusinessAccount: Pada rekening B001 dengan saldo awal Rp 2.000.000, penarikan sebesar Rp 1.500.000 berhasil ditolak (false) karena sisa saldo sebesar Rp 500.000 berada di bawah batas saldo minimum wajib rekening bisnis yaitu Rp 1.000.000.

Uji Overloading Setoran: Pemanggilan deposit(50000, "Initial top-up") berhasil mencetak log catatan setoran dan meneruskan penambahan dana ke saldo rekening, sehingga saldo meningkat menjadi Rp 150.000.

Pembuktian Polimorfisme Dinamis (Dynamic Method Dispatch): Saat Bank.printAllAccounts() memanggil acc.printInfo() melalui perulangan pada array of Account[], Java Runtime Environment (JVM) secara dinamis memilih dan mengeksekusi method printInfo() milik objek riil di heap (SavingsAccount, CheckingAccount, atau BusinessAccount). Masing-masing rekening berhasil mencetak format baris induk diikuti baris tipe spesifik rekeningnya masing-masing secara otomatis tanpa perlu branching (if/else).

Jawaban Tugas Mandiri:

1. Implementasi BusinessAccount:

Kelas BusinessAccount telah selesai diimplementasikan sesuai instruksi (Gambar 4). Bukti penarikan saldo minimum Rp 1.000.000 dan pengujian method printInfo() telah diverifikasi melalui eksekusi Main.java (Gambar 6). Hasil pengujian menunjukkan penarikan Rp 1.500.000 dari saldo Rp 2.000.000 menghasilkan 'Withdraw 1500000 allowed? false' dan saldo tetap Rp 2.000.000.

2. Jawaban Pertanyaan Konseptual:

(a) Perbedaan signature antara method yang di-override dan method yang di-overload:

Signature pada method yang di-override (seperti canWithdraw(double) dan printInfo()) harus persis sama antara subclass dan superclass (meliputi nama method, jumlah, urutan, serta tipe parameter). Sebaliknya, pada method yang di-overload (seperti deposit(double) dan deposit(double, String)), nama method sama persis namun daftar parameternya wajib berbeda (berbeda jumlah atau tipe data parameter).

(b) Penentu versi method yang dijalankan pada overriding vs overloading:

Pada method overriding, versi method yang dijalankan ditentukan secara dinamis saat runtime oleh Java Virtual Machine (JVM) berdasarkan tipe objek aktual yang diinstansiasi di memori heap (Late Binding / Polimorfisme Dinamis). Sedangkan pada method overloading, versi method ditentukan secara statis saat waktu kompilasi (compile-time) oleh compiler Java berdasarkan argumen yang diberikan pada pemanggilan method (Early Binding / Polimorfisme Statis).

Kesimpulan

Dari praktikum Jobsheet 7 ini, dapat disimpulkan bahwa Method Overriding dan Method Overloading merupakan dua pilar utama dalam mewujudkan konsep Polimorfisme pada pemrograman berorientasi objek di Java. Overriding memungkinkan kelas anak (subclass) untuk mengubah atau menyempurnakan perilaku bawaan kelas induk sesuai kebutuhan khususnya tanpa mengubah antarmuka umum (general interface), didukung oleh mekanisme dynamic dispatch saat runtime. Sementara itu, Overloading memberikan fleksibilitas tinggi bagi pemanggil method dengan menyediakan beragam opsi masukan (parameter) untuk fungsi yang memiliki nama tujuan serupa. Penerapan keduanya secara harmonis menghasilkan rancangan arsitektur perangkat lunak yang bersih, terstruktur, mudah dirawat, dan sangat modular.

Lampiran

Link Repositori GitHub untuk Source Code Lengkap:

https://github.com/helfa25/PRAKTIKUM-PBO.git